1 Síťový protokol HTTP

V této kapitole se dozvíte:

- Jaké jsou základní vlastnosti protokolu HTTP.
- Co je to URI.
- Jaké jsou metody požadavku strany klienta.
- Jaké odpovědi může generovat strana serveru.

Po jejím prostudování byste měli být schopni:

- Znát základní vlastnosti protokolu HTTP.
- Znát strukturu URI nebo URL identifikátoru.
- Rozlišovat jednotlivé metody požadavku strany klienta.
- Poznat, zda bylo zpracování požadavku na serveru úspěšné a jaký je jeho výsledek.

Klíčová slova této kapitoly:

HTTP, HTML, klient, server, síťový protokol, URL, URI.

Doba potřebná ke studiu: 4 hodiny

Průvodce studiem



Pro studium této kapitoly je vhodné znát alespoň základní pojmy z oblasti počítačových sítí. Studium kapitoly není příliš náročné, seznamuje čtenáře se základními vlastnostmi protokolu HTTP, s URI adresami, s metodami požadavku strany klienta a s druhy odpovědí strany serveru.

Nejznámější a nejpoužívanější službou mezinárodní počítačové sítě Internet je služba World Wide Web (WWW). Mezi další služby Internetu pak patří například FTP, e-mail, instant messaging (IM), VoIP nebo DNS. Služba WWW kombinuje text, grafiku a ostatní multimédia a propojuje je pomocí hypertextových odkazů. Pojmy WWW a Internet bývají někdy používány jako ekvivalentní, ale ve skutečnosti je WWW pouze jednou ze služeb Internetu.

Služba WWW je realizována pomocí speciálních programů, kdy mezi nejznámější patří WWW prohlížeč (MS Internet Explorer, Firefox, Opera, Google Chrome atd.) a WWW server (Apache, MS Internet Information Server atd.). WWW prohlížeč bývá nejčastěji umístěn na straně pracovní stanice a v komunikaci vystupuje v roli klienta. WWW server pak plní roli serveru služby WWW. Komunikace mezi klientem a serverem je zajišťována pomocí protokolu HTTP – HyperText Transfer Protocol.

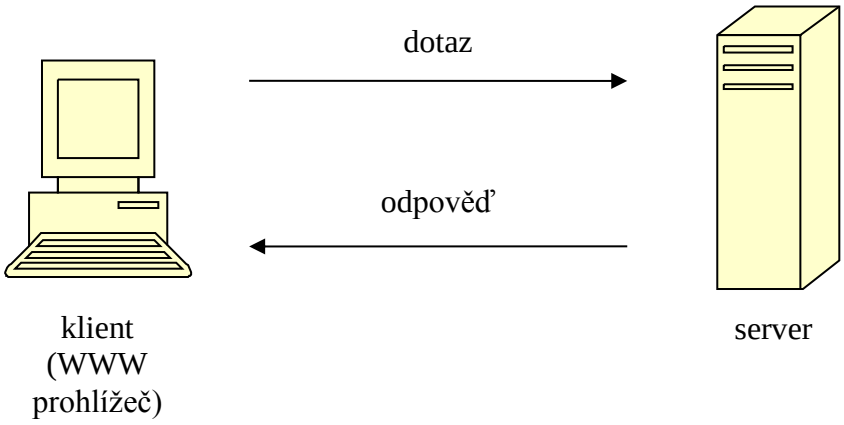
1.1 Protokol HTTP a jeho vlastnosti.

Protokol HTTP se poprvé objevil kolem roku 1990. Protokol původně vznikl v organizaci CERN (Evropská organizace pro jaderný výzkum), která sídlí

v Ženevě. Předchůdcem protokolu HTTP byl protokol Gopher, který se dnes již nepoužívá. Velmi významná byla verze 0.9 protokolu HTTP. V dnešní době se používá protokol HTTP ve verzi 1.1 z roku 1999, která je specifikovaná v RFC-2616 [1]. Protokol HTTP, stejně jako vše týkající se webu je dnes zaštiťován konsorciem W3C (www.w3.org).

Protokol HTTP je jedním z aplikačních protokolů TCP/IP. Své služby poskytuje nad transportním protokolem TCP, při jeho použití je komunikace spolehlivá a potvrzovaná. Pro použití protokolu HTTP je standardně používán port s číslem 80.

Komunikace pomocí protokolu HTTP je založena na architektuře klient-server. Klient, který bývá obvykle reprezentován WWW prohlížečem, odešle přesně strukturovaný požadavek na server. Server na základě požadavku zašle klientovi odpověď. V případě, že odeslání odpovědi není možné, odešle server chybové hlášení. Obecně platí, že pomocí protokolu HTTP lze přenášet libovolný obsah odpovídající doporučení MIME – Multipurpose Internet Mail Extension [2]. Nejčastěji bývá odpověď ve formě WWW stránky vytvořené v jazyce HTML [3]. Kromě toho ale může být odpovědí PDF soubor, obrázek, zvukový soubor, video soubor atd.



Obrázek 1.1 Komunikace klient-server.

Komunikace nejčastěji probíhá tak, že uživatel napíše do adresního řádku WWW prohlížeče URI identifikátor (adresu) objektu, který chce prohlížet. Prohlížeč nejprve z identifikátoru vyřízne jméno serveru a pomocí DNS jej přeloží na IP adresu. Poté klient naváže s touto IP adresou spojení pomocí protokolu TCP. Do takto vytvořeného kanálu vloží prohlížeč HTTP dotaz a server na něj reaguje odesláním HTTP odpovědi ve stejném kanálu. Nakonec zobrazí prohlížeč odpověď uživateli. V případě, že se výsledná webová stránka

skládá z více objektů, je nutné každý objekt stáhnout jedním HTTP dotazem. Ve starších verzích protokolu HTTP pro každý dotaz vždy navazovalo nové TCP spojení. Prvním dotazem se stáhl základní text stránky, který na místech objektů (obrázků atd.) obsahoval odkazy a poté se postupně stahovaly jednotlivé objekty. V protokolu HTTP v1.1 se již implicitně předpokládá, že TCP spojení mezi klientem a serverem bude navázáno pro celou sadu dotazů.

Odpověď serveru může být statická nebo dynamická. V případě statické odpovědi klient požaduje konkrétní soubor, který se nachází v adresářové struktuře serveru. V tom případě odešle server klientovi obsah tohoto souboru. Dynamická odpověď může být dvojího typu:

- Dynamická odpověď generovaná na straně serveru – v tomto případě obvykle server komunikuje s externí aplikací, která na základě parametrů zadaných klientem vygeneruje obsah odpovědi. Nejčastěji je výsledkem dynamicky vygenerovaná WWW stránka, která je poté serverem odeslaná klientovi.

- Dynamická odpověď generovaná na straně klienta – v tomto případě server vrací klientovi WWW stránku, která obsahuje dynamické prvky, např. Java Applet. Běh dynamického prvku je pak realizován v prohlížeči na straně klienta.

Komunikace protokolem HTTP tedy probíhá následujícím způsobem:

1. HTTP klient naváže prostřednictvím protokolu TCP komunikační spojení se stranou HTTP serveru.

2. Klient odešle požadavek (dotaz) na server.

3. Server zpracuje požadavek klienta a odešle odpověď, která má obecně multimediální charakter. V případě, že z důvodu nějaké chyby není možné odeslat klientovi požadovanou odpověď, server odešle klientovi chybovou zprávu.

4. Během jednoho spojení je možné poslat více dotazů a odpovědí.

5. Spojení mezi klientem a serverem je ukončeno.

1.2 URI



K identifikaci objektů ve webovém prostředí se používají tzv. URI (Uniform Resource Identifier) identifikátory. Speciálnějším případem URI je URL (Uniform Resource Locator) [3], které jsou v praxi používanější. Obecný tvar URL je:

```
schéma://server[:port][cesta[?dotaz#[fragment]]]
```

kde

- `schéma` – označuje použité schéma. V našem případě zde bude uvedeno http. Kromě toho zde může být ftp, mailto, telnet atd.
- `server` – označuje jedinečné jméno serveru, na který je odesílán požadavek ze strany klienta. Server může být identifikován svým doménovým jménem nebo IP adresou.
- `port` – číslo služby v rozsahu 0 až 65535, nad kterou je na daném serveru aktivní aplikace. Jak již bylo uvedeno, standardně je při komunikaci protokolem HTTP používán port s číslem 80. Pokud chceme použít tento port, není potřeba jej uvádět.
- `cesta` – označuje cestu v adresářové struktuře serveru, která určuje, jaký soubor je požadován. Začíná znakem lomítka „`/`“
- `dotaz` – zde jsou uvedeny parametry dotaz, může jich být několik. Parametr se zadává ve formě

  ```
  jmeno_parametru=hodnota_parametru
  ```

  Pokud je potřeba použít více parametrů jsou odděleny znakem „`&`“. Hodnota parametru se zadává bez uvozovek nebo apostrofů.
- `fragment` – začíná znakem „`#`“ a určuje konkrétní umístění na výsledné webové stránce. Po načtení stránky se okno prohlížeče přesune na umístění fragmentu.

Části URL identifikátoru umístěné mezi znaky „`[`“ a „`]`“ jsou nepovinné.

Příklady:

1. `http://www.osu.cz`



URL představuje požadavek klienta na obsah kořenového adresáře serveru *www.osu.cz*.

2. `http://www.osu.cz:8080/studium/index.html`

   URL obsahuje specifikaci portu, jedná se o port s číslem *8080*. Klient požaduje stránku *index.html*, která se nachází v adresáři *studium*.

3. `http://www.osu.cz/index.php?kategorie=10&id=1`

   URL obsahuje dva parametry. První parametr s názvem *kategorie* má hodnotu *10*, druhý parametr s názvem *id* má hodnotu *1*.

4. `http://www.osu.cz/index.php?kategorie=1#odstavec3`

   URL obsahuje jeden parametr s názvem *kategorie*, který má hodnotu *1*, a fragment s názvem *odstavec3*.

Pokud URI nezačíná názvem schématu, hovoříme o tzv. relativním URI. Relativní URI je vždy vztaženo k nějakému absolutnímu URI – k nějaké bázi. Bází může být URI aktuálního dokumentu, předešlého dokumentu nebo může být báze nastavena implicitně pro celou aplikaci. Relativní URI může ve své cestě obsahovat znak „`.`“ pro reprezentaci aktuálního adresáře nebo znak „`..`“ pro reprezentaci nadřízeného adresáře.

Příklady:

Mějme soubor s adresou *http://www.osu.cz/adresar/soubor.htm*. Tento soubor může obsahovat relativní odkazy:

- *#odstavec1* – tomuto odkazu odpovídá absolutní URI *http://www.osu.cz/adresar/soubor.htm#odstavec1*
- *soubor2.htm* – tomuto odkazu odpovídá absolutní URI *http://www.osu.cz/adresar/soubor2.htm*
- *../soubor3.htm* – tomuto odkazu odpovídá absolutní UR*I* *http://www.osu.cz/soubor3.htm*

1.3 Metody požadavku strany klienta



Požadavek strany klienta, kterým se odesílá HTTP dotaz na stranu serveru má pevně danou strukturu, viz. obrázek.

Metoda	`GET / HTTP/1.1`
Hlavičky požadavku	`Host: www.osu.cz` `Connection: keep-alive`
Prázdný řádek	
Data (volitelná část)	`kategorie=10&id=10`

Obrázek 1.2 Struktura HTTP dotazu.

První částí je specifikace metody požadavku. Dále následuje záhlaví, které je tvořeno jednotlivými hlavičkami. Na konci každé hlavičky je vždy umístěn konec řádku CRLF. Za poslední hlavičkou je vložen prázdný řádek, opět pomocí CRLF. Poslední částí jsou přenášená data. Tato část je volitelná.

Metoda

Protokol HTTP ve verzi 1.1 podporuje metody GET, POST, HEAD, OPTIONS, TRACE, PUT a DELETE. Každý WWW server musí povinně podporovat metody GET a HEAD. Ostatní metody nemusí být podporován. Specifikace metody požadavku je vždy ve tvaru

```
Metoda_požadavku URI_požadavku Protokol/Verze
```

Jako `Metoda_požadavku` může být uvedena jakákoliv z metod podporovaných protokolem HTTP. `URI_požadavku` představuje relativní cestu ke zdroji (WWW stránce), který klient požaduje. V poslední části bude vždy uvedeno `HTTP/1.1`.

Metoda GET slouží k získání statických zdrojů ze strany serveru, které mohou být obecně multimediálního charakteru. Těmito zdroji mohou být statické WWW stránky, obrázky, zvukové soubory atd. Metodu GET je možné použít i pro získání dynamického obsahu a to jak na straně serveru, tak na straně klienta. Obecně platí, že veškeré adresy, které klient napíše do adresního řádku WWW prohlížeče se odesílají metodou GET. Stejně tak se metodou GET odesílají všechny hypertextové odkazy, tj. vše, co je v HTML kódu webové stránky označeno tagem `<a> … </a>`. Důležitou vlastností metody GET je, že veškeré zdroje získané touto metodou jsou bookmarkovatelné. Uživatel si

může adresu zdroje uložit a navštívit ji znovu. Pokud jsou součástí URL i parametry, je možné navštívit odkaz se stejnými parametry. To může být za určitých okolností výhodou. Nevýhodou je, že délka URL je omezená, většinou je to 255 znaků. Další nevýhodou je, že parametry, které jsou součástí URL, jsou viditelné. Používat metodu GET pro přihlašování, kdy by adresa byla např.

```
http://www.stranka.cz/index.php?jmeno=pepa&heslo=123
```

by tedy nebylo zrovna bezpečné.

Metoda HEAD je velmi podobná metodě GET a stejně jako metoda GET musí být povinně implementována každým WWW serverem. Hlavním rozdílem mezi těmito metodami je, že metoda HEAD vrací pouze hlavičku odpovědi, al nevrací tělo odpovědi. Metoda HEAD se používá zejména v případech, kdy potřebujeme ověřit, zda požadovaný zdroj vůbec existuje, jaký má formát atd.

Další metodou je metoda POST, která bývá implementována prakticky na každém WWW serveru, i když patří mezi nepovinné metody. Společně s metodou GET patří mezi nejpoužívanější metody. Metoda POST slouží jednak získání dynamické odpovědi ze strany serveru a také k odesílání dat na server (upload na server). Na rozdíl od metody GET, nejsou dotazy odesílané metodou POST bookmarkovatelné. Samozřejmě je možné si uložit URL dotazu, který se odesílá metodou post, ale při zadání této adresy do prohlížeče může být výsledek dotazu úplně jiný nebo nemusí URL vůbec fungovat. Výhodou této metody ovšem je, že parametry odesílané na server nejsou viditelné v URL, ale jsou ukryty v těle dotazu. Takže zejména v případě odesílání citlivých dat je tato metoda výhodnější. Druhou výhodou je, že objem dat odesílaných na server není prakticky omezen. Na server je možné odesílat (uploadovat) soubory o velikosti několika GB (giga bajtů).

Metoda PUT slouží k uložení statických informací na WWW server. Při použití této metody je součástí dotazu URI zdroje, který má být uložen na server. Z bezpečnostních důvodů ale tato metoda nebývá povolena. Provozovatel WWW serveru určitě nemá zájem na tom, aby mohl kdokoliv ukládat soubory do adresářů na serveru.

Metoda DELETE umožňuje smazání zdroje, jehož URI je specifikováno v HTTP dotazu. Stejně jako metoda PUT bývá z bezpečnostních důvodů zakázána.

Metoda OPTIONS se používá ke zjištění komunikačních vlastností serveru nebo požadovaného URI. Pokud chceme zjistit vlastnost celého serveru, uvedeme místo URI znak hvězdičky ‚*'.

Metoda TRACE se používá v případě, kdy komunikace mezi klientem a serverem probíhá pomocí proxy serverů nebo bran. Odpovědí na dotaz metodou TRACE pak je seznam mezilehlých systémů (proxy serverů nebo bran), pomocí nichž byla provedena komunikace.

Hlavičky požadavku

Hlavičky požadavku obsahují jeden nebo více řádků, které poskytují serveru dodatečné informace o požadavku. Každý řádek hlavičky je ve tvaru:

```
Klíčové_slovo: Hodnota
```

Seznam klíčových slov je pevně dán. Povinná je pouze hlavička označená klíčovým slovem *Host*. Tato hlavička určuje adresu WWW serveru, na který je odesílán dotaz a dále určuje port, pomocí něhož je dotaz odesílán. Standardní port s číslem 80 nemusí být uveden. Ostatní hlavičky jsou nepovinné. Mezi nejpoužívanější hlavičky patří následující:

- *Accept* – určuje MIME typy, které smí strana klienta akceptovat, např. *text/html*.
- *Accept-Charset* – určuje znakové sady, které smí klient použít, např. *ISO-8859-2*.
- *Accept-Language* – záhlaví určuje, jaké jazyky akceptuje strana klienta, hodnotou záhlaví jsou standardní jazykové kódy podle specifikace RFC 1766, např. *en*, *en-us* atd.
- *Connection* – pomocí tohoto záhlaví je možné určit, že klient požaduje persistentní spojení protokolem HTTP, tj. že při navázání jediného spojení je možné obdržet více WWW stránek. Pro vytvoření persistentního spojení se používá hodnota *keep-alive*.
- *Content-Length* – záhlaví udává velikost datové části požadavku v bytech, používá se pouze u metody POST.

- *Content-Type* – záhlaví se používá spíše v odpovědi ze strany serveru, ale je možné ho použít i v požadavku. Používá se zejména při požadavcích metodou POST a PUT.

- *Cookie* – záhlaví se používá pro odeslání hodnot cookies na server.

- *If-Modified-Since* – záhlaví se používá v případě, kdy klient požaduje zaslání WWW stránky pouze v případě, pokud se změnila po uvedeném datu. Toto záhlaví umožňuje prohlížečům ukládat si stránky do paměťové cache a nenačítat je vždy znovu ze serveru.

- *Pragma* – toto záhlaví může mít pouze jednu hodnotu a tou je *no-cache*. Tato hodnota určuje, že server má odeslat WWW stránku, i když je uložena v cache paměti prohlížeče.

- *Referer* – záhlaví udává URL adresu WWW stránky, kterou klient požaduje. Většina WWW prohlížečů toto záhlaví odesílá v hlavičce požadavku.

- *User-Agent* – záhlaví slouží k identifikaci WWW prohlížeče nebo jiného typu klienta, např. Firefox, MS Internet Explorer atd.

Příklady HTTP dotazů



1. Dotaz metodou GET na kořenový adresář serveru *www.osu.cz*

```
GET / HTTP/1.1
Host: www.osu.cz
Accept: text/html, application/xhtml+xml,
 application/xml
Accept-Encoding: gzip, deflate
Accept-Language: cs, en-us, en
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 6.1; rv:16.0)
 Gecko/20100101 Firefox/16.0
```

2. Dotaz metodou GET na adresu

   http://prf.osu.cz/kip/index.php?kategorie=502

```
GET /kip/index.php?kategorie=502 HTTP/1.1
Host: prf.osu.cz
```

```
Accept: text/html, application/xhtml+xml,
 application/xml
Accept-Encoding: gzip, deflate
Accept-Language: cs, en-us, en
Connection: keep-alive
```

3. Dotaz metodou POST na adresu

   http://www.osu.cz/index.php?kategorie=892

```
POST /index.php?kategorie=892 HTTP/1.1
Host: www.osu.cz
Accept: text/html, application/xhtml+xml,
 application/xml
Accept-Language: cs, en-us, en
Connection: keep-alive

zpravas=a&lidis=a&kategs=a&search=ahoj&go=Hledat
```

4. Dotaz metodou PUT, který realizuje vložení souboru test.html do adresáře /testovaci na serveru www.osu.cz

```
PUT /testovaci/test.html HTTP/1.1
Host: www.osu.cz
Connection: keep-alive
Content-Length: 50

<HTML>
  <HEAD>
    <TITLE>Testovací dotaz metodou PUT</TITLE>
  </HEAD>
  <BODY>
    Obsah WWW stránky
  </BODY>
</HTML>
```

5. Požadavek na zrušení souboru *test.html*, který se nachází ve složce */testovaci* na serveru *www.osu.cz*

```
DELETE /testovaci/test.html HTTP/1.1
Connection: keep-alive
Host: www.osu.cz
```

1.4 Odpověď strany serveru



Poté, co WWW server obdrží požadavek ze strany klienta, generuje odpověď na tento požadavek. Struktura odpovědi je podobná jako struktura požadavku strany klienta.

Stavový řádek	`HTTP/1.1 200 OK`

Hlavičky odpovědi	`Server: Apache`
	`Content-Type: text/html`
	`Connection: close`

Prázdný řádek	

Tělo odpovědi	`<html>`
	`<head>`
	`<title>Stránka</title>`
	`</head>`
	`<body>`
	`ahoj`
	`</body>`
	`</html>`

Obrázek 1.3 Struktura odpovědi strany serveru.

První částí odpovědi strany serveru je stavový řádek, který říká, zda bylo zpracování požadavku úspěšné či nikoliv. V případě neúspěšného zpracování požadavku je zde uveden popis chyby, která nastala. Další částí jsou hlavičky, které jsou velmi podobné hlavičkám HTTP dotazu. Za poslední hlavičkou následuje prázdný řádek. Poslední částí je Tělo odpovědi, které obsahuje samotný obsah odpovědi. V případě WWW stránky je zde uveden její HTML kód, v případě obrázku je to obrázek v daném formátu (JPEG, PNG, GIF).

Stavový řádek

Stavový řádek odpovědi ze strany serveru je vždy v následujícím tvaru:

`Protokol/Verze Kód_stavu Popis`

První část, `Protokol/Verze` bude v našem případě vždy `HTTP/1.1`. `Kód_stavu` je trojmístné přirozené číslo v rozsahu 100 – 599 a dělí se do pěti kategorií:

- 1xx – informativní stavové kódy, označující, že zpracování pokračuje dále.

- 2xx – stavové kódy označující úspěšné splnění požadavku ze strany klienta.

- 3xx – stavové kód pro přesměrování, další akce se bude týkat jiného URI.

- 4xx – stavové kódy pro identifikaci chyby na straně klienta, např. syntaktická chyba v dotazu, dotaz na neexistující WWW stránku.

- 5xx – stavové kódy pro identifikaci chyby na straně serveru, např. chyba v CGI skriptu.

Poslední část stavového řádku, `Popis`, pak obsahuje slovní popis stavu.

Hlavičky odpovědi

Hlavičky odpovědi jsou podobné hlavičkám požadavku, také obsahují jeden nebo více řádků, které poskytují klientovi dodatečné informace o odpovědi nebo o serveru samotném. Každý řádek hlavičky je ve tvaru:

```
Klíčové_slovo: Hodnota
```

Stejně jako v případě požadavku, i zde je seznam klíčových slov je pevně dán. Mezi nejpoužívanější záhlaví patří:

- *Allow* – záhlaví určuje, které metody požadavků podporuje WWW server. Toto záhlaví je vygenerováno, pokud je v požadavku použita metoda OPTIONS.

- *Cache-control* – záhlaví určuje, zda a za jakých podmínek smí klient uložit WWW stránku do své cache paměti. Přípusné hodnoty jsou *public, private, no-cache, no-store, must-revalidate, proxy-revalidate, max-age=xxx, s-max-age=xxx* (*xxx* označuje čas v sekundách)

- *Connection* – pokud je hodnotou tohoto záhlaví slovo close, je tím klient informován, že persistentní konektivita byla ukončena.

- *Content-Language* – záhlaví udává, v jakém jazyce je dokument odeslaný klientovi jako odpověď. Hodnotami jsou, stejně jako v případě požadavku, standardní jazykové kódy dle RFC 1766, např. *en, en-us* atd.

- *Content-Length* – záhlaví udává velikost odpovědi v bytech, která byla odeslána klientovi. Tato hlavička je povinná v případě, že má být zachováno persistentní spojení.

- *Content-Type* – hodnotou záhlaví je MIME typ odpovědi, která byla odeslána klientovi. Je vždy ve tvaru *maintype/subtype* (pro registrované typy) a *maintype/x-subtype* (pro neregistrované typy). Příkladem hodnoty tohoto záhlaví je *text/html*, *image/png* atd.

- *Last-Modified* – záhlaví udává datum a čas poslední změny dokumentu.

- *Location* – toto záhlaví je povinné v případě, že kód stavu odpovědi je 3xx. Hodnotou záhlaví je adresa dokumentu, na kterou vede přesměrování.

- *Pragma* – stejně jako v případě požadavku, i zde udává hodnota *no-cache* informaci, že dokument nemá být uložen do cache paměti prohlížeče.

- *Server* – záhlaví obsahuje informaci o identifikaci WWW serveru, hodnotou může být např. *Apache*.

- *Set-Cookie* – záhlaví specifikuje cookie, které souvisí s danou WWW stránkou

Příklady odpovědí strany serveru

1. Odpověď při úspěšném zpracování požadavku



```
HTTP/1.1 200 OK
Content-Type: text/html
Server: Apache
Connection: close
Pragma: no-cache
Date: Wed, 10 Apr 2013 13:19:05 GMT

<HTML>
… Obsah WWW stránky …
</HTML>
```

2. Odpověď generující kód 301

```
HTTP/1.1 301 Moved Permanently
```

```
Server: Apache
Connection: close
Location: http://prf.osu.cz/kip/

<HTML/>
  Obsah WWW stránky
</HTML>
```

3. Odpověď generující chybovou zprávu s kódem 404 – stránka nenalezena

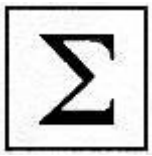
```
HTTP/1.1 404 Not Found
Server: Apache
Connection: close

<HTML/>
  Obsah chybové WWW stránky
</HTML>
```



Kontrolní otázky:

1. Jaký je princip komunikace klient-server?
2. Jaké jsou metody požadavků protokolu HTTP?
3. Jaká je struktura požadavku strany klienta?
4. Jaká je struktura odpovědi strany serveru?



Shrnutí obsahu kapitoly

V této kapitole jste se seznámili s protokolem HTTP, který se používá při komunikaci v rámci služby WWW. Základní vlastnosti tohoto protokolu, struktura URI adresy (identifikátoru), metody požadavku strany klienta a druhy odpovědi strany serveru jsou důležité znalosti potřebné pro tvorbu webových aplikací.